

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_6237b110-d58\agent-a1bbc5f5c7e7d201b.jsonl`  
- SHA-256: `2c81057c0fcce50460eaf3302a268136f3d4ff88ac22fd6e4d0c8d1ff9e114ba`  
- Source modified: `2026-06-06T17:41:32+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_6237b110-d58`  
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T17:37:47.648Z)

Adversarial Claim Verifier (voter 3/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Research alternative methods to automatically detect rally start/end segments (temporal segmentation) in racket-sport video ? especially badminton ? for a SELF-HOSTED, LOCAL-FIRST pipeline, and assess whether a cloud-VLM teacher ? local-model distillation is the best approach or whether other methods are competitive or complementary. Keep it focused ("mini" research): prioritize actionable, commercially-licensable, locally-runnable approaches over exotic SOTA needing huge compute or restrictive licenses.

OUR CONTEXT / CONSTRAINTS (ground every recommendation in these):

- Self-hosted on a single consumer GPU (RTX 2070 Super), Windows + WSL2. We want LOCAL-ONLY inference at runtime ? privacy lever: user video never leaves the device. It's a commercial product, so PERMISSIVE / commercial-friendly licenses are REQUIRED (avoid AGPL and non-commercial-only weights/datasets).
- Current substrate: WASB (MIT) shuttle-trajectory detector ? per-frame (x, y, visibility). We derive rally windows via velocity/merge-gap/min-duration thresholds plus a net-crossing gate. We ALSO have a permissive person detector available (torchvision Faster R-CNN + supervision ByteTrack) and the video's AUDIO track.
- Current best approach: use paid Gemini (cloud VLM, full-context) OFFLINE as a TEACHER to label rallies, then DISTILL into a local model (logistic regression on shuttle-trajectory features). Findings so far: threshold-tuning ceilings around F1 ? 0.44 and overfits with few videos; the learned model is data-starved at n=2 labeled videos; WASB candidate RECALL is a real bottleneck (its windows sometimes don't align with true rallies at IoU 0.5).
- Inputs we can exploit locally: video frames, the shuttle trajectory, player bounding boxes / pose (if we add a permissive pose model), and the AUDIO track (shuttle "thwack" hits, crowd noise, umpire/score calls).

SPECIFICALLY COVER:

1. Method families for rally/point segmentation in racket sports (badminton, tennis, table tennis, pickleball): audio-based shot/hit detection; temporal action localization (TAL) deep models; pose/player-stance + court-homography state machines (e.g., "serve-ready stance" gating); classical change-point detection / HMM on the shuttle trajectory; scoreboard / score-overlay OCR for point boundaries; self-supervised / weakly-supervised approaches; and VLM teacher?student distillation.
2. For EACH family: how it works, the labels/data it needs, expected accuracy, local-inference COMPUTE cost, and LICENSE / commercial viability of any models, datasets, or weights involved.
3. Badminton rally-segmentation datasets (e.g., ShuttleSet, TrackNet/TrackNetV2 data) and their licenses / commercial usability.
4. A RANKED recommendation for our local-first, permissive, single-GPU setup: is Gemini-teacher ? local-distillation the best, or are some alternatives competitive or complementary ?

especially AUDIO-based hit detection and pose/court-geometry state machines, which we can run fully locally?

5. Concrete, cheap next experiments we could run with what we already have (shuttle trajectory + person detector + audio).

Claim under review

"The paper presents a deep-learning pipeline that includes rally-wise video trimming (a form of temporal rally segmentation) using shot-angle recognition, achieving 99% accuracy on shot angle recognition for that trimming step."

****Source:**** <https://arxiv.org/pdf/2307.16000> (primary)

****Supporting quote:**** "we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection."

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

****refuted=true**** if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.

****refuted=false**** ONLY if: claim is well-supported, current, and source quality matches claim strength.

Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-06-06T17:37:51.682Z)

I'll verify this claim about the arXiv paper. Let me fetch the source and analyze it skeptically.

3. assistant (2026-06-06T17:37:53.958Z)

Let me fetch the paper to examine the actual claims and methodology.

4. user (2026-06-06T17:38:00.587Z)

I cannot extract the requested information from the provided content. The text appears to be a raw PDF stream containing compressed/encoded data rather than readable document text. The content shows PDF object definitions with binary and compressed content that cannot be meaningfully parsed for claims about video trimming, shot angle recognition, accuracy metrics, or bibliographic details.

To answer your questions accurately, I would need:

- A readable text version of the paper, or
- A properly decompressed PDF, or
- The paper title/authors to locate the actual document

****Recommendation:**** Please provide the document in a text-readable format (plain text, uncompressed PDF, or HTML) so I can identify and quote the specific claims about rally-wise video trimming, shot angle recognition methods, and accuracy percentages.

[Binary content (application/pdf, 1.5MB) also saved to C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\tool-results\webfetch-1780766977975-dnjzkl.pdf]

5. assistant (2026-06-06T17:38:02.638Z)

The PDF was downloaded. Let me read it directly and also try the abstract page.

6. user (2026-06-06T17:38:04.184Z)

pdftoppm failed: Command 'pdftoppm' not found or is in an unsafe location (current directory)

7. user (2026-06-06T17:38:16.449Z)

Paper Details

Title: Automated Hit-frame Detection for Badminton Match Analysis

Authors: Yu-Hang Chien and Fang Yu

Abstract (verbatim):

"Sports professionals constantly under pressure to perform at the highest level can benefit from sports analysis, which allows coaches and players to reduce manual efforts and systematically evaluate their performance using automated tools. This research aims to advance sports analysis in badminton, systematically detecting hit-frames automatically from match videos using modern deep learning techniques. The data included in hit-frames can subsequently be utilized to synthesize players' strokes and on-court movement, as well as for other downstream applications such as analyzing training tasks and competition strategy. The proposed approach in this study comprises several automated procedures like rally-wise video trimming, player and court keypoints detection, shuttlecock flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection."

Key Results:

- The researchers achieved "99% accuracy on shot angle recognition for video trimming"
- They obtained "over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction"
- The study involved automated detection of hit-frames using techniques including rally-wise video trimming, keypoint detection, and shuttlecock trajectory prediction

8. assistant (2026-06-06T17:38:19.387Z)

I have the abstract verbatim. Now let me dig into the methodology to understand what "rally-wise video trimming" and "shot angle recognition" actually mean, and whether the 99% is for temporal segmentation or something narrower. Let me get the HTML version and search for context.

9. user (2026-06-06T17:38:22.963Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

10. user (2026-06-06T17:38:29.878Z)

Web search results for query: "Automated Hit-frame Detection for Badminton" "shot angle recognition" "rally-wise video trimming" method"

Links: [{"title": "[2307.16000] Automated Hit-frame Detection for Badminton Match Analysis", "url": "https://arxiv.org/abs/2307.16000"}, {"title": "Automated Hit-frame Detection for Badminton Match Analysis - ADS", "url": "https://ui.adsabs.harvard.edu/abs/2023arXiv230716000C/abstract"}, {"title": "GitHub - arthur900530/Automated-Hit-frame-Detection-for-Badminton-Match-Analysis", "url": "https://github.com/arthur900530/Automated-Hit-frame-Detection-for-Badminton-Match-Analysis"}, {"title": "Automated Hit-frame Detection for Badminton Match Analysis | DeepAI", "url": "https://deepai.org/publication/automated-hit-frame-detection-for-badminton-match-"}]

```
analysis"}],{"title":"Automated Hit-frame Detection for Badminton Match
Analysis","url":"https://arxiv.org/pdf/2307.16000"}, {"title":"Badminton Video Analysis based on
Spatiotemporal and Stroke Features","url":"https://www.researchgate.net/publication/317159692_Ba
dminton_Video_Analysis_based_on_Spatiotemporal_and_Stroke_Features"}, {"title":"Automated Hit-
frame Detection for Badminton Match Analysis |
alphaXiv","url":"https://www.alphaxiv.org/abs/2307.16000"}, {"title":"[PDF] Badminton shot
classification in compressed video with baseline angled camera | Semantic
Scholar","url":"https://www.semanticscholar.org/paper/Badminton-shot-classification-in-
compressed-video-Careelmont/d47f36358d50977ab5d3377d0b3f0b6177ae9f01"}, {"title":"Enhancing
Badminton Game Analysis: An Approach to Shot Refinement via a Fusion of Shuttlecock Tracking and
Hit Detection from Monocular Camera","url":"https://www.mdpi.com/1424-8220/24/13/4372"}]
```

Based on the search results, I found information about the "Automated Hit-frame Detection for Badminton" research you're looking for:

Overview

This research aims to advance sports analysis in badminton, systematically detecting hit-frames automatically from match videos using modern deep learning techniques.

Key Method Components

The proposed approach comprises several automated procedures like rally-wise video trimming, player and court keypoints detection, shuttlecock flying direction prediction, and hit-frame detection.

Performance Results

The study achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction.

Specific Contributions

The main contributions include utilizing the interchange of shot angle to trim rally-wise frames from raw badminton video. They also proposed a novel transformer that predicts shuttlecock direction sequences based on player keypoint sequence.

Applications

The data included in hit-frames can subsequently be utilized to synthesize players' strokes and on-court movement, as well as for other downstream applications such as analyzing training tasks and competition strategy.

The research appears to have been published in 2023, and an official implementation is available on GitHub at the repository "Automated-Hit-frame-Detection-for-Badminton-Match-Analysis."

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

11. assistant (2026-06-06T17:38:33.025Z)

The search result is highly revealing. The key phrase: "utilizing the **interchange of shot angle** to trim rally-wise frames from raw badminton video." This tells me what "shot angle recognition" really means. Let me confirm by reading the paper's methodology section via the alphaXiv or HTML version.

12. user (2026-06-06T17:38:36.700Z)

The server returned HTTP 403 Forbidden.

The response body was not retrieved. If this URL requires authentication, use an authenticated

tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

13. user (2026-06-06T17:38:42.747Z)

Rally-wise Video Trimming and Shot Angle Recognition

Based on the repository documentation, the shot angle recognition module is designed to `***trim rally-wise frames from raw badminton video***` by utilizing "the interchange of shot angle."

However, the provided content does not include specific details about:

- Whether "shot angle" refers to camera view classification (court vs. non-court perspectives)
- Which specific model architecture performs shot angle recognition
- The accuracy metrics for this component

The repository description indicates this is a novel approach to `***bridge the gap between raw badminton videos and analyzable data,***` but the webpage excerpt doesn't contain the technical specifications needed to answer your questions about the shot angle module's mechanics or performance. For those details, you would need to consult the full paper (referenced as arXiv:2307.16000) or the source code in the repository.

14. assistant (2026-06-06T17:38:45.740Z)

Let me read the GitHub repo source to definitively understand the "shot angle" mechanism, and try the ADS abstract for the methodology.

15. user (2026-06-06T17:40:27.785Z)

Cloning into 'badminton-paper'...

```
---LS---
total 798
drwxr-xr-x 1 avidu 197609    0 Jun  6 23:10 .
drwxr-xr-x 1 avidu 197609    0 Jun  6 23:10 ..
drwxr-xr-x 1 avidu 197609    0 Jun  6 23:10 .git
-rw-r--r-- 1 avidu 197609  108 Jun  6 23:10 .gitattributes
-rw-r--r-- 1 avidu 197609  725 Jun  6 23:10 .gitignore
-rw-r--r-- 1 avidu 197609   526 Jun  6 23:10 CITATION.cff
-rw-r--r-- 1 avidu 197609  1091 Jun  6 23:10 LICENSE
-rw-r--r-- 1 avidu 197609  2256 Jun  6 23:10 README.md
drwxr-xr-x 1 avidu 197609    0 Jun  6 23:10 configs
drwxr-xr-x 1 avidu 197609    0 Jun  6 23:10 outputs
drwxr-xr-x 1 avidu 197609    0 Jun  6 23:10 src
drwxr-xr-x 1 avidu 197609    0 Jun  6 23:10 videos
```

16. user (2026-06-06T17:40:30.614Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

17. user (2026-06-06T17:40:31.268Z)

===LICENSE===

MIT License

Copyright (c) 2023 Yu-Hang Chien

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

```
===SRC TREE===
src/ai_coach.py
src/eval.ipynb
src/main.py
src/models/layers.py
src/models/sacnn.py
src/models/transformer.py
src/models/weights/sacnn.pt
src/models/weights/scaler.pickle
src/models/__init__.py
src/param_parser.py
src/rally_processor.py
src/utils/eval_utils.py
src/utils/utils.py
src/utils/__init__.py
src/video_resolver.py
```

18. assistant (2026-06-06T17:40:34.169Z)

The repo is MIT licensed. There's a `sacnn.py` model ("SACNN" = Shot Angle CNN) and a `rally\_processor.py`. Let me read these to understand exactly what "shot angle" means.

19. user (2026-06-06T17:40:34.675Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

20. user (2026-06-06T17:40:35.074Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

21. assistant (2026-06-06T17:40:37.533Z)

The files are in /tmp (WSL/bash filesystem), not the Windows cwd. Let me read them via Bash.

22. user (2026-06-06T17:40:39.075Z)

```
=====sacnn.py=====
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import transforms

class SACNN(nn.Module):
    def __init__(self):
        super(SACNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=1)
        self.pool1 = nn.MaxPool2d(kernel_size=(2, 2))
        self.bn1 = nn.BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=1)
```

```

        self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))
        self.bn2 = nn.BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        self.conv3 = nn.Conv2d(64, 32, kernel_size=(3, 3), stride=(1, 1), padding=1)
        self.pool3 = nn.MaxPool2d(kernel_size=(2, 2))
        self.bn3 = nn.BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        self.l1 = nn.Linear(27*27*32, 2)
        self.dropout = nn.Dropout(p=0.1)

```

```

def forward(self, x):
    x = self.bn1(self.pool1(self.conv1(x)))
    x = F.relu(x)
    x = self.bn2(self.pool2(self.conv2(x)))
    x = F.relu(x)
    x = self.bn3(self.pool3(self.conv3(x)))
    x = F.relu(x)
    x = x.view(-1, 27*27*32)
    x = self.l1(x)
    x = F.relu(x)
    out = self.dropout(x)
    return out

```

```

class SACNNContainer(object):
    def __init__(self, args):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.args = args
        self.setup_model()

    def setup_model(self):
        self.model = SACNN().to(self.device)
        self.model.load_state_dict(torch.load(self.args['sacnn_path']))
        self.model.eval()

    def predict(self, batch, processed=False):
        batch = self.preprocess(batch) if not processed else batch
        out = self.model(batch)
        predicted = torch.argmax(out, dim=1)
        return predicted.item()

```

```

def preprocess(self, img):
    pre = transforms.Compose([
        transforms.Resize((216, 384)),
        transforms.CenterCrop((216, 216)),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])
    img = pre(img).to(self.device)
    batch = torch.unsqueeze(img, 0)
    return batch

```

```

=====video_resolver.py (head)=====
from models.sacnn import SACNNContainer
from rally_processor import RallyProcessor
from utils.utils import get_path, check_dir
from PIL import Image
import cv2
import json

```

```

class ShotAngleQueue(object):
    def __init__(self, max_len):
        self.max_len = max_len
        self.queue = []
        self.last_sa = 0

```

```

def push(self, frame_info):
    sa_condition = None
    if len(self.queue) < self.max_len:          # queue is not full
        self.queue.append(frame_info)
        return None, None
    else:
        first_info = self.queue.pop(0)
        sa, sa_condition = self.__check_sa_conditon(first_info[0])
        self.last_sa = sa
        first_info[0] = sa

        self.queue.append(frame_info)

        return first_info, sa_condition

```

```

def get(self, index):
    return self.queue[index]

```

```

def __check_sa_conditon(self, sa):
    '''
    return sa, cond in {0, 1, 2, 3}
    cond : last sa -> sa
    0 :      0 -> 0
    1 :      0 -> 1
    2 :      1 -> 1
    3 :      1 -> 0
    '''
    sum = 0 + sa
    if self.last_sa == 1 and sa == 0:
        for info in self.queue:
            sum += info[0]
        if sum <= (self.max_len / 2):
            return 0, 3
        else:
            return 1, 2
    elif self.last_sa == 0 and sa == 1:
        for info in self.queue:
            sum += info[0]
        if sum >= (self.max_len / 2):
            return 1, 1
        else:
            return 0, 0
    elif self.last_sa == 1 and sa == 1:
        return 1, 2
    elif self.last_sa == 0 and sa == 0:
        return 0, 0

```

```

class VideoResolver(object):
    def __init__(self, args):
        self.args = args
        self.model_args = self.args['model']
        self.__setup_videos()
        self.__setup_rally_processor()
        self.__sacnn = SACNNContainer(self.model_args)

    def start_resolve(self):
        for path in self.__video_paths:
            self.video_name = path.split('/')[-1].split('.mp4')[0]
            resolved = self.__build_saving_directories()
            if not resolved:
                self.__setup()
                self.__resolve(path)
            else:
                print(f'{self.video_name} was resolved.')

```

```

def __setup(self):
    self.__setup_sa_queue()
    self.__rally_processor.reset()

def __build_saving_directories(self):
    output_dirs = [self.args['video_save_path'],
                    self.args['joint_save_path'],
                    self.args['rally_save_path'],
                    f"{self.args['video_save_path']}/{self.video_name}",
                    f"{self.args['joint_save_path']}/{self.video_name}",
                    f"{self.args['rally_save_path']}/"]

    resolved = True
    for dir in output_dirs:
        resolved_dir = check_dir(dir)
        if not resolved_dir:
            resolved = False
    self.video_save_path = output_dirs[3]
    self.joint_save_path = output_dirs[4]
    self.rally_save_path = output_dirs[5]
    return resolved

def __setup_sa_queue(self):
    self.__sa_queue = ShotAngleQueue(self.args['sa_queue length'])

def __setup_rally_processor(self):
    self.__rally_processor = RallyProcessor(self.model_args)

def __setup_videos(self):
    self.__video_paths = get_path(self.args['video_directory'])

def __create_video(self, out, drawn_img_list):
    for i in range(len(drawn_img_list)):
        out.write(drawn_img_list[i])

def __resolve(self, vid_path):
    vid_name = vid_path.split('/')[1].split('.')[0]
    cap = cv2.VideoCapture(vid_path)
    frame_width = int(cap.get(3))
    frame_height = int(cap.get(4))
    total_frame_count = int(cap.get(7))
    fps = cap.get(5)

```

23. assistant (2026-06-06T17:40:45.913Z)

This is now definitive. The `SACNN` is a binary classifier (final layer `nn.Linear(..., 2)` ? 2 classes, argmax over 2). The `ShotAngleQueue` tracks `sa` values of `{0, 1}` ? i.e., "shot angle" is a **binary scene/camera-view classification**: is the current frame the main court-view shot angle (game play, `sa=1`) or not (`sa=0`, e.g. replays/closeups/crowd). The "interchange of shot angle" (0?1, 1?0 transitions in the queue) is what marks rally start/end and trims the video into rally clips.

So the 99% accuracy is the **binary frame-classification accuracy** of this small CNN* (court-view vs not), NOT a temporal segmentation IoU on rally boundaries. Let me confirm the paper explicitly distinguishes the 99% (shot angle recognition) from the rally-trimming evaluation, and check the abstract wording once more against the claim. Let me look at the eval notebook and the paper's evaluation framing.

24. user (2026-06-06T17:40:47.991Z)

```

=====rally_processor.py (head 60)=====
import torch
import torchvision
import numpy as np
import copy
import cv2

```

```

from PIL import Image
from torchvision.transforms import transforms
from torchvision.transforms import functional as F
from models.transformer import OptimusPrimeContainer

```

```

class RallyProcessor(object):
    '''
    Tasks involving Keypoint RCNNs
    '''
    def __init__(self, args):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.args = args
        self.got_info = False
        self.rally_count = 0
        self.__setup_sotrage_lists()
        self.__setup_RCNN()
        self.__setup_opt()

    def reset(self):
        self.got_info = False
        self.rally_count = 0
        self.__setup_sotrage_lists()
        self.rally_info = None

    def __setup_RCNN(self):
        self.__court_kpRCNN = torch.load(self.args['court_kpRCNN_path'])
        self.__court_kpRCNN.to(self.device).eval()
        self.__kpRCNN = torch.load(self.args['kpRCNN_path'])
        self.__kpRCNN.to(self.device).eval()

    def __setup_opt(self):
        self.__opt = OptimusPrimeContainer(self.args)

    def __setup_sotrage_lists(self):
        self.drawn_img_list = []
        self.player_joint_list = []
        self.frame_num_list = []
        self.start_end_frame_list = []

    def get_court_info(self, img, frame_height):
        img = F.to_tensor(img)
        img = img.unsqueeze(0)
        img = img.to(self.device)
        output = self.__court_kpRCNN(img)
        scores = output[0]['scores'].detach().cpu().numpy()
        high_scores_idx = np.where(scores > 0.7)[0].tolist()
        post_nms_idx = torchvision.ops.nms(output[0]['boxes'][high_scores_idx],
                                           output[0]['scores'][high_scores_idx],
                                           0.3).cpu().numpy()
        keypoints = []
        for kps in
output[0]['keypoints'][high_scores_idx][post_nms_idx].detach().cpu().numpy():
            keypoints.append([list(map(int, kp[:2])) for kp in kps])

        self.__true_court_points = copy.deepcopy(keypoints[0])

```

====grep 99 / accuracy / shot angle in repo====

```

./README.md:4:  - Utilize the interchange of shot angle to trim rally-wise frames from raw
badminton video.
./src/utls/eval_utils.py:51:    accuracy, precision, recall, f1 = cal_confusion_matrix(tp, fp,
fn, tn)
./src/utls/eval_utils.py:58:    print(f'Accuracy: {accuracy:.4f}')
./src/utls/eval_utils.py:67:    accuracy = round((tp + tn) / (tp + fn + fp + tn), 4)

```

```

./src/utlils/eval_utils.py:71:     return accuracy, precision, recall, f1
./src/utlils/eval_utils.py:74: def rally_wise_video_trimming_eval(trim_output_dir,
eval_target_dir):
./src/utlils/eval_utils.py:78:     extra_trimmed = 0
./src/utlils/eval_utils.py:81:     total_trimmed_rallies = 0
./src/utlils/eval_utils.py:82:     correctly_trimmed_ranges = {}
./src/utlils/eval_utils.py:84:     trim_output_paths = get_path(trim_output_dir)
./src/utlils/eval_utils.py:87:     for i in range(len(trim_output_paths)):
./src/utlils/eval_utils.py:89:         correctly_trimmed_list = []
./src/utlils/eval_utils.py:90:         output_name =
trim_output_paths[i].split('/')[ -1].split('.json')[0]
./src/utlils/eval_utils.py:103:         with open(trim_output_paths[i]) as file:
./src/utlils/eval_utils.py:104:             trimmed_rallies = json.load(file)
./src/utlils/eval_utils.py:105:             trimmed_rallies = trimmed_rallies['rally']
./src/utlils/eval_utils.py:106:             total_trimmed_rallies += len(trimmed_rallies)
./src/utlils/eval_utils.py:108:             copied_rallies = copy.deepcopy(trimmed_rallies)
./src/utlils/eval_utils.py:110:             for trimmed_rally in trimmed_rallies:
./src/utlils/eval_utils.py:111:                 trimmed_start = int(trimmed_rally[0])
./src/utlils/eval_utils.py:112:                 trimmed_end = int(trimmed_rally[1])
./src/utlils/eval_utils.py:118:                 if trimmed_start - 3 < true_start < true_end <
trimmed_end + 3:
./src/utlils/eval_utils.py:119:                     correctly_trimmed_list.append((true_start,
true_end))
./src/utlils/eval_utils.py:122:                     copied_rallies.remove(trimmed_rally)
./src/utlils/eval_utils.py:125:                     elif true_start < trimmed_start < true_end <
trimmed_end:
./src/utlils/eval_utils.py:126:                         correctly_trimmed_list.append((trimmed_start,
true_end))
./src/utlils/eval_utils.py:129:                         copied_rallies.remove(trimmed_rally)
./src/utlils/eval_utils.py:132:                         elif trimmed_start < true_start < trimmed_end <
true_end:
./src/utlils/eval_utils.py:133:                             correctly_trimmed_list.append((true_start,
trimmed_end))
./src/utlils/eval_utils.py:136:                             copied_rallies.remove(trimmed_rally)
./src/utlils/eval_utils.py:138:                             correctly_trimmed_ranges[output_name] =
correctly_trimmed_list
./src/utlils/eval_utils.py:140:                     extra_trimmed += len(copied_rallies)
./src/utlils/eval_utils.py:144:     accuracy, precision, recall, f1 =
cal_confusion_matrix(correctly_predicted, extra_trimmed, missed)
./src/utlils/eval_utils.py:147:     print('Extra Trimmed: ', extra_trimmed)
./src/utlils/eval_utils.py:150:     print('Total Trimmed: ', total_trimmed_rallies)
./src/utlils/eval_utils.py:153:     print(f'Accuracy: {accuracy:.4f}')
./src/utlils/eval_utils.py:159:     return correctly_trimmed_ranges
./src/utlils/eval_utils.py:296:     accuracy_0, precision_0, recall_0, f1_0 =
cal_confusion_matrix(d_zero['tp'], d_zero['fp'], d_zero['fn'], d_zero['tn'])
./src/utlils/eval_utils.py:297:     accuracy_1, precision_1, recall_1, f1_1 =
cal_confusion_matrix(d_one['tp'], d_one['fp'], d_one['fn'], d_one['tn'])
./src/utlils/eval_utils.py:298:     accuracy_2, precision_2, recall_2, f1_2 =
cal_confusion_matrix(d_two['tp'], d_two['fp'], d_two['fn'], d_two['tn'])

```

25. assistant (2026-06-06T17:40:56.572Z)

This is decisive evidence. The repo has a **separate evaluation function** `rally_wise_video_trimming_eval` (lines 74-159 of eval_utils.py) that computes its own accuracy/precision/recall/F1 for rally trimming ? distinct from the shot-angle classifier accuracy. This confirms that:`

1. **"Shot angle recognition"** = a binary CNN frame classifier (court main-view vs not). The 99% is this classifier's accuracy.
2. **"Rally-wise video trimming"** is a *downstream* step driven by shot-angle interchange, and it has its OWN evaluation (not 99%).

Now let me assess the claim wording precisely. The claim states: "achieving 99% accuracy on shot angle recognition for that trimming step." Let me re-read the abstract quote against the claim's exact wording to judge whether the claim is faithful or an overreach.

The abstract says verbatim: `"we achieved 99% accuracy on shot angle recognition for video trimming"`. The claim says: `"rally-wise video trimming ... using shot-angle recognition, achieving 99% accuracy on shot angle recognition for that trimming step."`

Let me verify the date and that the rally-trimming eval numbers are NOT 99% (to confirm the 99% is specifically the shot-angle classifier, not the segmentation quality).

26. user (2026-06-06T17:40:58.288Z)

```
===CITATION.cff===
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
authors:
- family-names: "Chien"
  given-names: "Yu-Hang"
  orcid: "https://orcid.org/0000-0002-7334-327X"
- family-names: "Yu"
  given-names: "Fang"
  orcid: "https://orcid.org/0000-0002-2776-9624"
title: "Automated-Hit-frame-Detection-for-Badminton-Match-Analysis"
version: 1.0.0
doi: 10.5281/zenodo.1234
date-released: 2023-07-23
url: "https://github.com/arthur900530/Automated-Hit-frame-Detection-for-Badminton-Match-Analysis"
```

```
===eval_utils 40-75 (shot angle eval + cal_confusion)===
    'sacnn_path': './models/weights/sacnn.pt'
}
sacnn = SACNNContainer(args)

tp = 0
fp = 0
fn = 0
tn = 0
for inputs, labels in dataloaders['val']:
    inputs = inputs.to(device)
    labels = labels.to(device)
    output = sacnn.predict(inputs, processed=True)
    if output == labels.item() == 1:
        tp += 1
    if output == 1 and labels.item() == 0:
        fp += 1
    if output == labels.item() == 0:
        tn += 1
    if output == 0 and labels.item() == 1:
        fn += 1

accuracy, precision, recall, f1 = cal_confusion_matrix(tp, fp, fn, tn)
print('=====')
print(f'SA-CNN Evaluation:')
print('-----')
print(f'Training Data Count: {len(dataloaders["train"])}')
print(f'Testing Data Count: {len(dataloaders["val"])}')
print('-----')
print(f'Accuracy: {accuracy:.4f}')
print(f'Precision: {precision:.4f}')
print(f'Recall: {recall:.4f}')
print(f'F1-score: {f1:.4f}')
print('=====')
```

```
def cal_confusion_matrix(tp, fp, fn, tn=None):
    tn = tn if tn else 0
    accuracy = round((tp + tn) / (tp + fn + fp + tn), 4)
```

```
precision = round(tp / (tp + fp), 4)
recall = round(tp / (tp + fn), 4)
f1 = round(2 / ((1 / recall) + (1 / precision)), 4)
return accuracy, precision, recall, f1
```

```
def rally_wise_video_trimming_eval(trim_output_dir, eval_target_dir):
    gotcha = 0
```

27. user (2026-06-06T17:41:02.641Z)

I don't see any web page content provided in your message. The section marked "Web page content:" appears to be empty.

Could you please share the actual webpage or paper content you'd like me to analyze? Once you provide it, I'll be happy to identify the publication date and peer-review status while following your specified guidelines.

28. assistant (2026-06-06T17:41:19.241Z)

I have everything I need. The evidence is now fully assembled and decisive.

****Findings confirmed from primary source (paper repo + verbatim abstract):****

1. ****The 99% is the SA-CNN classifier accuracy**** ? a binary court-view-vs-not frame classifier (`SACNN`, final `Linear(...,2)`), evaluated as a confusion matrix on a held-out frame set (`SA-CNN Evaluation` block). "Shot angle recognition" = scene/camera-view classification, NOT geometric shot-direction and NOT temporal boundary IoU.

2. ****Rally-wise video trimming is a separate, downstream step**** with its OWN distinct evaluation function (`rally\_wise\_video\_trimming\_eval`) reporting separate accuracy/precision/recall/F1. The abstract itself separates them: it "achieved 99% accuracy on shot angle recognition for video trimming" and **separately** "reported the evaluation results of rally-wise video trimming." The 99% is explicitly NOT the trimming-quality number.

3. It IS genuinely a form of temporal rally segmentation (shot-angle interchange 0?1 marks rally start/end and cuts broadcast into rally clips), so the claim's framing of "rally-wise video trimming (a form of temporal rally segmentation)" is fair.

****Assessment of the claim:**** The claim states the pipeline "includes rally-wise video trimming... using shot-angle recognition, achieving 99% accuracy on shot angle recognition for that trimming step." This faithfully mirrors the abstract's exact wording ("99% accuracy on shot angle recognition for video trimming"). The claim is careful: it attributes 99% to "shot angle recognition," not to the trimming/segmentation IoU. It does not overclaim that rally boundary detection is 99% accurate.

Now the skeptical voter's verdict. My job is to try to refute. The strongest refutation angle would be "99% is a classifier accuracy, not segmentation quality ? so it's misleading." But the claim itself correctly scopes the 99% to "shot angle recognition," matching the source verbatim. So that refutation doesn't land ? the claim is accurately scoped. Source is the primary paper (arXiv, MIT-licensed official repo). It is a non-peer-reviewed preprint (2023), which is a minor quality caveat, but the claim is a descriptive characterization of what the paper itself reports, not an extraordinary scientific claim needing independent replication.

The claim is well-supported by the verbatim quote and the source code. It is not refuted.

29. user (2026-06-06T17:41:27.007Z)

Structured output provided successfully

30. assistant (2026-06-06T17:41:32.651Z)

Verdict: ****NOT refuted**** (refuted=false), high confidence.

The claim faithfully reflects the primary source. The verbatim abstract states "99% accuracy on shot angle recognition for video trimming," and the claim correctly scopes the 99% to shot-angle recognition rather than overstating it as rally-boundary segmentation quality. Source-code inspection of the official MIT-licensed repo confirms SACNN is a binary court-view-vs-not frame classifier, that shot-angle interchange (0?1) drives the rally trimming, and that rally-wise trimming has its own separate evaluation ? so the paper itself keeps the 99% distinct from trimming quality, exactly as the claim represents it.

Relevant local artifact for the parent agent: the cloned repo at `/tmp/badminton-paper` (key files: `src/models/sacnn.py`, `src/video\_resolver.py`, `src/utils/eval\_utils.py`, `LICENSE` = MIT).